

2. AR Vision

2.1. Overview

Augmented Reality (AR) is a technology that ingeniously blends virtual information with the real world. It extensively employs various technical means—including multimedia, 3D modeling, real-time tracking and registration, intelligent interaction, and sensing—to simulate and apply computer-generated virtual information (such as text, images, 3D models, music, and video) onto the real world. These two types of information complement each other, thereby achieving an "augmentation" of the real world.

AR systems possess three prominent characteristics: ① Integration of information from both the real and virtual worlds; ② Real-time interactivity; and ③ The addition of localized virtual objects within a 3D spatial environment.

Augmented Reality technology encompasses a range of new technologies and techniques, including multimedia, 3D modeling, real-time video display and control, multi-sensor fusion, real-time tracking and registration, and scene fusion.

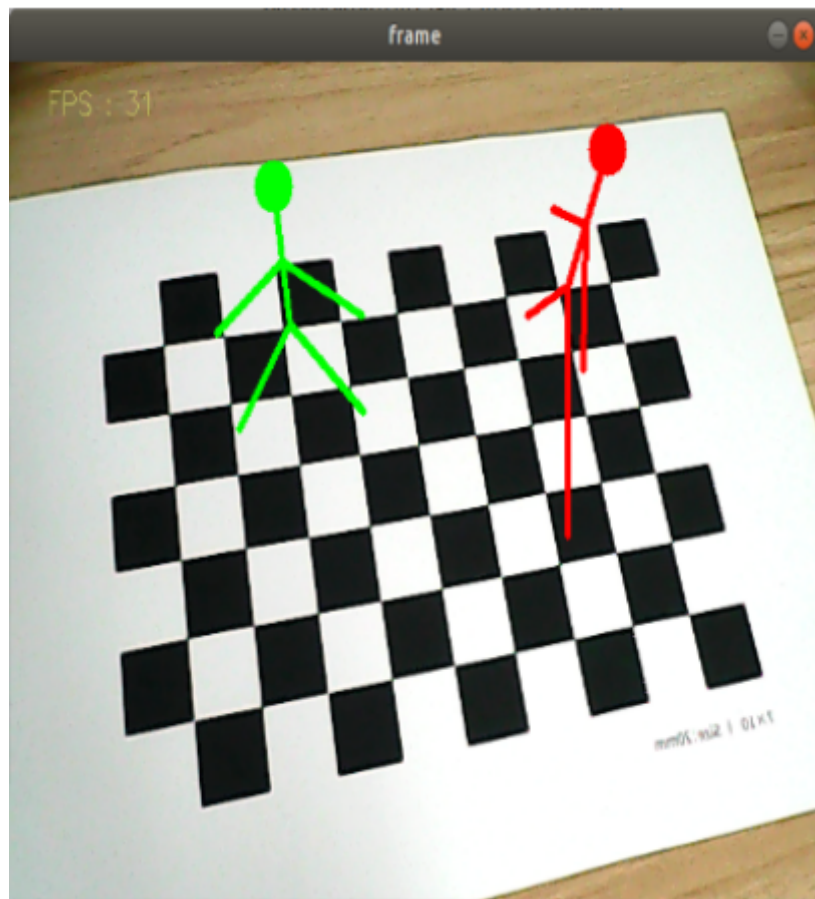
2.3. Usage Instructions

Launch the Monocular Camera

```
ros2 launch dofbot_info camera_arm_kin.launch.py
```

Launch the AR Vision Node

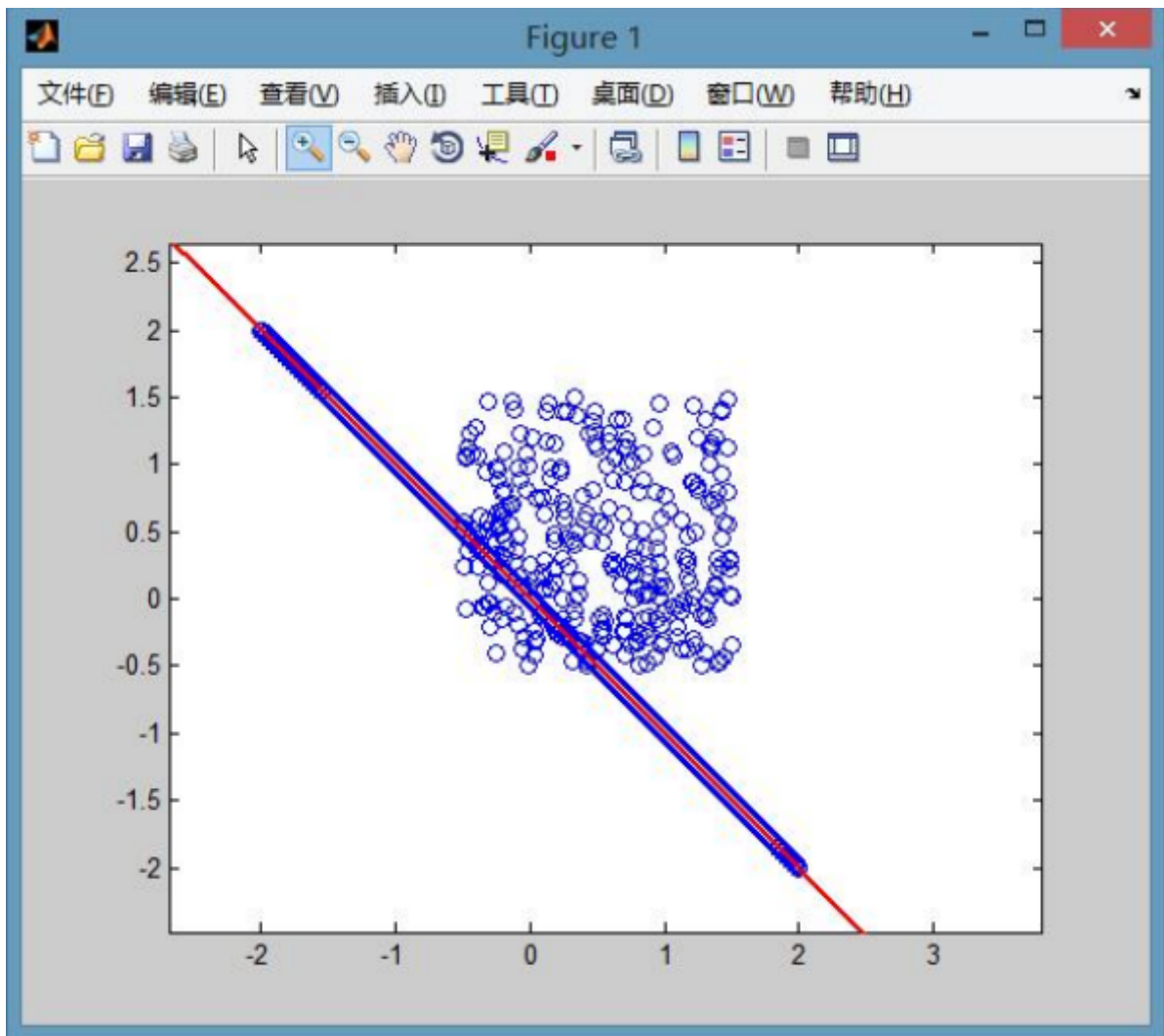
```
ros2 run yahboom_vision simple_ar
```



2.3.1. RANSAC Approach

- Algorithm Principle: Uses the RANSAC approach to determine an object's pose based on 3D-to-2D point correspondences.

The RANSAC (Random Sample Consensus) algorithm is a classic algorithm originally developed for data processing; its function is to extract specific components of an object or dataset even in the presence of significant noise. The figure below illustrates the effectiveness of the RANSAC algorithm. In the diagram, a set of points clearly aligns with a specific straight line, while another cluster of points constitutes pure noise. The objective is to determine the equation of the straight line amidst this substantial noise—in this particular example, the volume of noisy data is three times greater than that of the points defining the line.



If the Least Squares method were applied instead, it would not yield this result; the resulting line would instead be positioned slightly higher than the actual line shown in the figure. - The fundamental assumptions of RANSAC are:

- The data consists of "inliers"—i.e., data points whose distribution can be explained by a specific set of model parameters;
- "Outliers" are data points that do not fit this model;
- Any data falling outside these categories is considered noise.
- Outliers can arise from various sources, such as: extreme values within the noise distribution; erroneous measurement techniques; or incorrect assumptions regarding the data. RANSAC also posits the following assumption: given a set of inliers (typically a very small subset), there exists a procedure capable of estimating the model parameters, such that the resulting model can explain—or is applicable to—those inliers.

2.3.2. Source Code

Source Code Location:

/home/yahboom/dofbot_ws/src/yahboom_vision/yahboom_vision/simple_ar.py

```
import os
import sys
import time
import cv2 as cv
import numpy as np
from cv_bridge import CvBridge

# ros2 lib
```

```

# ROS2库
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from sensor_msgs.msg import CompressedImage, Image

print("import done")

cv_edition = cv.__version__
print("cv_edition: ", cv_edition)

class simple_AR(Node):
    # Simple Augmented Reality class for displaying graphics based on chessboard
    pose
    # 简单增强现实类，用于根据棋盘格姿态显示图形

    def __init__(self, name):
        # Initialize the ROS2 node with the given name
        # 使用给定的名称初始化ROS2节点
        super().__init__(name)

        # Create a publisher for the processed camera image
        # 创建用于发布处理后相机图像的发布者
        self.pub_img = self.create_publisher(Image, '/simpleAR/camera', 1)

        # Create a subscriber for receiving graphics selection commands
        # 创建用于接收图形选择命令的订阅者
        self.sub_graphics = self.create_subscription(
            String, '/Graphics_topic', self.choose_Graphics, 1)

        # Flag to flip the image horizontally
        # 水平翻转图像的标志
        self.flip = True

        # Index for cycling through graphics
        # 用于循环切换图形的索引
        self.index = 0

        # List of available graphics shapes
        # 可用图形形状列表
        self.graphics = ["Triangle", "Rectangle", "Parallelogram", "windMill",
                        "TableTennisTable", "Ball", "Arrow", "Knife", "Desk",
                        "Bench", "Stickman", "ParallelBars"]

        # Currently selected graphic
        # 当前选中的图形
        self.Graphics = self.graphics[self.index]

        # 3D axis points for pose estimation (in object coordinate system)
        # 用于姿态估计的3D轴点（在物体坐标系中）
        self.axis = np.float32([
            [0, 0, -1], [0, 8, -1], [5, 8, -1], [5, 0, -1],
            [1, 2, -1], [1, 6, -1], [4, 2, -1], [4, 6, -1],
            [1, 0, -4], [1, 8, -4], [4, 0, -4], [4, 8, -4],
            [1, 2, -4], [1, 6, -4], [4, 2, -4], [4, 6, -4],
            [0, 1, -4], [3, 2, -1], [2, 2, -3], [3, 2, -3],
            [1, 2, -3], [2, 2, -4], [2, 2, -5], [0, 4, -4],
            [2, 3, -4], [1, 3, -4], [4, 3, -5], [4, 5, -5],

```

```

        [1, 2, -3], [1, 6, -3], [5, 2, -3], [5, 6, -3],
        [3, 4, -5], [0, 6, -4], [5, 6, -4], [2, 8, -4],
        [3, 8, -4], [2, 6, -4], [2, 0, -4], [1, 5, -4],
        [3, 0, -4], [3, 2, -4], [0, 3, -4], [1, 2, -4],
        [4, 2, -4], [5, 3, -4], [2, 7, -4], [3, 7, -4],
        [3, 3, -1], [3, 5, -1], [1, 5, -1], [1, 3, -1],
        [3, 3, -3], [3, 5, -3], [1, 5, -3], [1, 3, -3],
        [1, 3, -6], [1, 5, -6], [3, 3, -4], [3, 5, -4],
        [0, 0, -4], [3, 1, -4], [1, 1, -4], [0, 2, -4],
        [2, 4, -4], [4, 4, -4], [0, 8, -4], [5, 8, -4],
        [5, 0, -4], [0, 4, -5], [5, 4, -4], [5, 4, -5],
        [2, 5, -1], [2, 7, -1], [2, 6, -3], [2, 6, -5],
        [2, 5, -3], [2, 7, -3]
    ])

    # Current video frame
    # 当前视频帧
    self.frame = None

    # Image window name
    # 图像窗口名称
    self.img_name = 'img'

    # Chessboard pattern size (internal corners)
    # 棋盘格图案尺寸（内角点数量）
    self.patternSize = (6, 9)

    # CV bridge for ROS2 <-> OpenCV conversion
    # 用于ROS2与OpenCV转换的CV桥接器
    self.bridge = CvBridge()

    # Camera calibration file path
    # 相机标定文件路径
    yaml_path =
'/home/yahboom/dofbot_ws/src/yahboom_vision/config/camera.yaml'

    # Load camera intrinsic parameters if calibration file exists
    # 如果标定文件存在，加载相机内参
    if os.path.exists(yaml_path):
        fs = cv.FileStorage(yaml_path, cv.FileStorage_READ)
        self.cameraMatrix = fs.getNode("camera_matrix").mat()
        self.distCoeffs = fs.getNode("distortion_coefficients").mat()
    else:
        # Use empty parameters if no calibration file
        # 如果没有标定文件，使用空参数
        self.distCoeffs, self.cameraMatrix = (), ()

    # Generate 3D object points for chessboard corners
    # 生成棋盘格角点的3D物体点
    self.objectPoints = np.zeros((6 * 9, 3), np.float32)
    self.objectPoints[:, :2] = np.mgrid[0:6, 0:9].T.reshape(-1, 2)

    # Subscribe to raw camera image topic
    # 订阅原始相机图像话题
    topic_name = '/image_raw'
    self.subscription = self.create_subscription(
        Image,
        topic_name,

```

```

        self.image_callback,
        10)

def choose_Graphics(self, msg):
    # Callback for graphics selection topic
    # 图形选择话题的回调函数
    if not isinstance(msg, String):
        return
    if msg.data in self.graphics:
        self.Graphics = msg.data
    else:
        self.graphics_update()

def graphics_update(self):
    # Cycle to the next graphic in the list
    # 循环切换到列表中的下一个图形
    self.index += 1
    if self.index >= len(self.graphics):
        self.index = 0
    self.Graphics = self.graphics[self.index]

def image_callback(self, msg):
    # Callback for receiving camera images
    # 接收相机图像的回调函数
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    action = cv.waitKey(10) & 0xFF
    frame = self.process(frame, action)
    cv.imshow('frame', frame)
    if action == ord('q') or action == 113:
        cv.destroyAllWindows()

def process(self, img, action):
    # Main image processing function
    # 主图像处理函数
    if self.flip == True:
        img = cv.flip(img, 1)
    if action == ord('f') or action == ord('F'):
        self.graphics_update()

    # Convert to grayscale for corner detection
    # 转换为灰度图以进行角点检测
    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

    # Find chessboard corners
    # 查找棋盘格角点
    retval, corners = cv.findChessboardCorners(
        gray, self.patternSize, None,
        flags=cv.CALIB_CB_ADAPTIVE_THRESH + cv.CALIB_CB_NORMALIZE_IMAGE +
cv.CALIB_CB_FAST_CHECK)

    # Refine corner locations to subpixel accuracy
    # 将角点位置精细化到亚像素精度
    if retval:
        corners = cv.cornerSubPix(
            gray, corners, (11, 11), (-1, -1),
            (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001))

    # Compute object pose using solvePnP

```

```

        # 使用solvePnPRansac计算物体姿态
        retval, rvec, tvec, inliers = cv.solvePnPRansac(
            self.objectPoints, corners, self.cameraMatrix, self.distCoeffs)

        # Project 3D axis points onto the image plane
        # 将3D轴点投影到图像平面
        image_Points, jacobian = cv.projectPoints(
            self.axis, rvec, tvec, self.cameraMatrix, self.distCoeffs)

        # Draw the selected graphic on the image
        # 在图像上绘制选中的图形
        img = self.draw(img, corners, image_Points)

        # Display current graphic name on the image
        # 在图像上显示当前图形名称
        cv.putText(img, self.Graphics, (240, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9,
            (0, 0, 255), 1)

        # Publish the processed image
        # 发布处理后的图像
        self.pub_img.publish(self.bridge.cv2_to_imgmsg(img, "bgr8"))
        return img

    def draw(self, img, corners, image_Points):
        # Draw different graphics based on selection
        # 根据选择绘制不同的图形
        # Note: OpenCV uses BGR color order
        # 注: OpenCV使用BGR颜色顺序
        img_pts = np.int32(image_Points).reshape(-1, 2)

        if self.Graphics == "Triangle":
            cv.drawContours(img, [np.array([img_pts[14], img_pts[15],
            img_pts[23]])], -1, (255, 0, 0), -1)
        elif self.Graphics == "Rectangle":
            cv.drawContours(img, [np.array([img_pts[12], img_pts[13],
            img_pts[15], img_pts[14]])], -1, (0, 255, 0), -1)
        elif self.Graphics == "Parallelogram":
            cv.drawContours(img, [np.array([img_pts[12], img_pts[10],
            img_pts[15], img_pts[9]])], -1, (65, 105, 225), 1)
        elif self.Graphics == "WindMill":
            cv.drawContours(img, [np.array([img_pts[60], img_pts[38],
            img_pts[61], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[10], img_pts[14],
            img_pts[58], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[62], img_pts[63],
            img_pts[23], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[25], img_pts[64],
            img_pts[65], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.line(img, tuple(img_pts[64]), tuple(img_pts[35]), (0, 255, 0), 3)
        elif self.Graphics == "TableTennisTable":
            cv.line(img, tuple(img_pts[0]), tuple(img_pts[60]), (255, 0, 0), 3)
            for i in range(1, 4):
                cv.line(img, tuple(img_pts[i]), tuple(img_pts[65 + i]), (255, 0,
            0), 3)

            cv.drawContours(img, [np.array([img_pts[60], img_pts[66],
            img_pts[67], img_pts[68]])], -1, (0, 255, 0), -1)
            cv.drawContours(img, [np.array([img_pts[23], img_pts[69],
            img_pts[71], img_pts[70]])], -1, (0, 0, 255), -1)

```

```

        elif self.Graphics == "Ball":
            cv.circle(img, tuple(img_pts[22]), 30, (0, 0, 255), -1)
        elif self.Graphics == "Arrow":
            cv.drawContours(img, [np.array([img_pts[13], img_pts[34],
img_pts[36]]), -1, (0, 255, 0), -1)
            cv.drawContours(img, [np.array([img_pts[37], img_pts[15],
img_pts[10], img_pts[38]]), -1, (0, 255, 0), -1)
        elif self.Graphics == "Knife":
            cv.drawContours(img, [np.array([img_pts[58], img_pts[24],
img_pts[35], img_pts[47]]), -1, (160, 252, 0), -1)
            cv.drawContours(img, [np.array([img_pts[40], img_pts[38],
img_pts[21], img_pts[41]]), -1, (30, 144, 255), -1)
            cv.drawContours(img, [np.array([img_pts[42:46]]), -1, (0, 0, 255),
-1)

        elif self.Graphics == "Desk":
            for i in range(4):
                cv.line(img, tuple(img_pts[4 + i]), tuple(img_pts[12 + i]),
(163, 148, 128), 3)
            cv.drawContours(img, [np.array([img_pts[14], img_pts[12],
img_pts[13], img_pts[15]]), -1, (0, 199, 140), -1)
        elif self.Graphics == "Bench":
            for i in range(4):
                cv.line(img, tuple(img_pts[48 + i]), tuple(img_pts[52 + i]),
(255, 0, 0), 3)
            cv.drawContours(img, [img_pts[52:56]], -1, (0, 0, 255), -1)
            cv.drawContours(img, [img_pts[54:58]], -1, (139, 69, 19), -1)
        elif self.Graphics == "Stickman":
            cv.line(img, tuple(img_pts[18]), tuple(img_pts[4]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[18]), tuple(img_pts[6]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[18]), tuple(img_pts[21]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[21]), tuple(img_pts[19]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[21]), tuple(img_pts[20]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[21]), tuple(img_pts[22]), (0, 0, 255), 3)
            cv.circle(img, tuple(img_pts[22]), 15, (0, 0, 255), -1)
            cv.line(img, tuple(img_pts[74]), tuple(img_pts[72]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[74]), tuple(img_pts[73]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[74]), tuple(img_pts[37]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[37]), tuple(img_pts[76]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[37]), tuple(img_pts[77]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[37]), tuple(img_pts[75]), (0, 255, 0), 3)
            cv.circle(img, tuple(img_pts[75]), 15, (0, 255, 0), -1)
        elif self.Graphics == "ParallelBars":
            for i in range(4):
                cv.line(img, tuple(img_pts[4 + i]), tuple(img_pts[12 + i]),
(255, 0, 0), 3)
            cv.line(img, tuple(img_pts[8]), tuple(img_pts[9]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[10]), tuple(img_pts[11]), (0, 0, 255), 3)
        return img

def main():
    print("start")
    rclpy.init()
    ar = simple_AR('simple_AR')
    rclpy.spin(ar)

```